함수

학습목표

- 함수를 정의하고 사용하는 방법을 익혀 함수를 정의할 수 있다
- 매개변수와 반환값의 개념을 파악하고 함수 내에서의 변수를 활용할 수 있다.
- 전역 변수와 지역 변수의 차이를 이해하고, 전역 변수를 함수 내에서 사용하여 프로그램을 작성할 수 있다
- 재귀함수의 원리를 이해하고 사용방법을 익혀 프로그램을 작성할 수 있다

목차

- 함수란?
- 함수만들기
- 전역변수(Global variable)를 활용한 함수
- 재귀함수(Recursive function)

9.1 함수란?

(1) 함수의 이해

- 특정한 작업을 위해 서로 관련되어 있는 일련의 명령문
- 함수를 정의할 때 이름을 지정하고, 그 함수가 수행할 일을 위해 순서대로 명령
- 함수의 이름으로 함수를 호출

함수의 예)

표준출력함수 print(), 표준입력함수 input(), 데이터타입을 확인하는 type() 함수, 길이를 반환해주는 len()함수, 리스트나 특정변수를 삭제하는 del()함수, 데이터 구조를 초기에 생성할 때 사용하는 list()함수, 범위를 지정하는 range()함수

9.1 함수란?

- 함수 사용의 이유

① 프로그램에서 좋은 코드란 중복성이 없는 코드를 말한다. 중복된 코드는 여러 번 사용한다는 것을 의미하기도 한다. 이 때, 중복된 코드를 별도로 분류하여 함수로 만들게 되면 **코드의 양을 줄일 수 있고**, 이로 인해서 **가독성**이 좋아진다.

② 둘째, 간결해진 코드는 **전체의 기능을 이해**하기에 수월하다.

③ 셋째, 프로그램의 흐름을 파악하기 쉽기 때문에 **관리하기 쉽다**.

④ 넷째, 중요한 기능만 함수로 정의해두면 이를 필요로 하는 프로그램이나 다른 프로그래머에게 **재사용**이 가능하여 **매우 효율적으로 사용**할 수 있다.

9.1 함수란?

(2) 함수의 사용

```
>>> type(32)
<class 'int'>
```

인수

반환값

- 함수의 이름은 type이다.
- 인수는 함수에 대한 입력으로 함수에 전달하는 값 또는 변수이다
- type함수의 결과는 인수의 자료형 타입이다
- 함수가 인수를 취하고, 결과를 반환한다고 말하는 것이 일반적이다
- 여기서 결과를 반환값(return value)라고 부른다.

9.2 함수 만들기

- 함수를 직접 만들기 위해서는 def라는 예약어를 사용한다

함수 기본 형식

```
def 함수 이름(매개변수) :
        명령문1
        ...
        return 반환값
함수 이름(인수) # 함수 호출
```

- 함수를 사용하려면 함수를 먼저 만들고 나서 함수를 호출하여 사용해야 한다

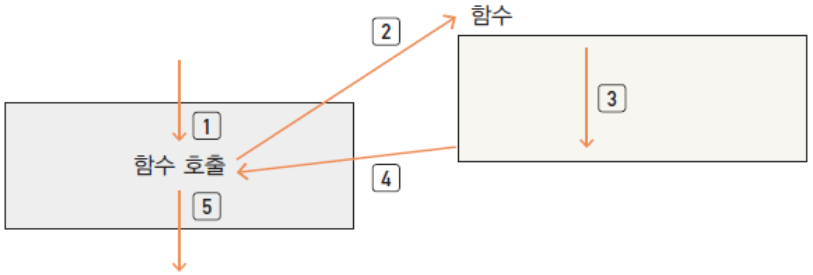


그림 9.2

9.2 함수 만들기

(1) 함수 이름

- 함수 이름을 작성할 때에도 변수에서 이름을 정의했던 것과 규칙이 같음

함수 이름 작성 예

```
def output( ) :
    print('python')
    ...
```

9.2 함수 만들기

(2) 매개변수(Parameter)

- 함수 이름 뒤에는 빈 괄호를 넣거나, 괄호 안에 인수를 입력 받는 함수로 작성할 수 있다
- 이때, 인수(argument)는 호출된 함수에 전달할 값이다
- 매개 변수(parameter)는 호출된 함수에서 전달 받은 값을 임시로 할당하는 변수이다

매개변수 0개	매개변수 1개	매개변수 2개

`def output() :`
`  print('python')`
`output()`
	`def output(s) :`
	`  print(s)`
	`output('python')`
		`def output(s1,s2) :`
		`  print(s1+s2)`
		`output('python','A')`

9.2 함수 만들기

(2) 매개변수(Parameter)

- 일반 매개 변수 : 인수의 개수 만큼 매개변수를 사용

일반 매개변수가 있는 함수 구조

```
def 함수 이름(매개변수1, 매개변수2) :
    명령문1
    ...
```

예제 9.1 매개변수가 있는 함수 사용 예

```
def add(x, y) :
    print(x+y)
add(11,12)
```

실행결과

```
23
```

! 일반 매개변수는 인수보다 많거나 적으면 오류가 발생한다

9.2 함수 만들기

(2) 매개변수(Parameter)

- 기본 매개 변수

기본 매개변수가 있는 함수 구조

```
def 함수 이름(매개변수, 매개변수, … .,매개변수=값):
    명령문1
    ...
```

예제 9.3 기본 매개변수 사용 예

```
def print_n(value,n=2):
  for i in range(n):
      print(value)
print_n('python')
```

실행결과

```
python
python
```

9.2 함수 만들기

(2) 매개변수(Parameter)

- 가변 매개 변수

가변 매개변수가 있는 함수 구조

```
def 함수 이름(매개변수, 매개변수, ··· ., *매개변수):
    명령문1
    ...
```

예제 9.4 가변 매개변수 예1

```
def print_n(n,*value):
  print(value*n)
print_n(2,'python','love')
```

실행결과

```
('python', 'love', 'python', 'love')
```

예제 9.5 가변 매개변수 예2

```
def print_n(*value):
  print(value)
print_n('python','love')
```

실행결과

```
('python', 'love')
```

9.2 함수 만들기

(2) 매개변수(Parameter)

- 키워드 매개 변수

키워드 매개변수가 있는 함수 구조

```
def 함수 이름(*매개변수1, 매개변수2=값):
    명령문1
    ...
```

예제 9.6 키워드 매개변수 예1

```
def print_n(*value,n=2):
  print(value*n)
print_n('python','love')
```

실행결과

```
('python', 'love', 'python', 'love')
```

예제 9.7 키워드 매개변수 예2

```
def print_n(*value,n=2):
  print(value*n)
print_n('python','love',n=1)
```

실행결과

```
('python', 'love')
```

9.2 함수 만들기

(3) 반환값(return value)

- 함수가 작업을 마친 후 호출한 곳으로 돌려주는 결과 값을 반환값(리턴값)이라 한다
- 모든 함수에서 반환값이 존재하지는 않는다
- return 명령어는 함수를 종료하고 함수를 실행했던 위치로 돌아가라는 뜻이다
- 반환값이 있는 함수는 return 명령어를 사용하여 결과값을 가지고 함수를 호출했던 위치로 돌아간다

9.2 함수 만들기

(3) 반환값(return value)

반환값이 있는 함수의 구조

```
def 함수 이름(매개변수1, 매개변수2) :
    ...
    return 값
```

예제 9.8 반환값이 있는 함수 사용 예

```
def add(x, y) :
    return x+y
save=add(10,11)
print(save)
```

실행결과

```
21
```

9.2 함수 만들기

(3) 반환값(return value)

여러 개의 반환값이 있는 함수 구조

```
def 함수 이름(매개변수1, 매개변수2) :
    명령문1
    ...
    return 값1, 값2
```

예제 9.9 여러 개의 리턴값이 있는 함수 사용 예

```
def add(x, y) :
    return x+y, x*y
save1,save2=add(10,11)
print(save1,save2)
```

실행결과

```
21 110
```

9.3 전역변수(Global variable) 활용 함수

- 전역변수란 함수 외부에서 선언된 변수로, 프로그램 전체에서 공동으로 사용할 수 있는 변수이다

전역변수를 사용하는 함수 구조

```
전역변수1=값
def 함수 이름() :
    전역변수1 사용
    ...
```

예제 9.10 전역변수를 사용하는 함수 예

```
x = 10 # 전역 변수 선언
def test():
  print(x) # 전역 변수 출력

test()
```

실행결과

```
10
```

9.3 전역변수(Global variable) 활용 함수

- 함수에서 전역변수에 접근하여 값을 변경하려면 global 명령어를 사용한다

전역변수를 변경하는 함수 구조

```
전역변수1=값
def 함수 이름() :
    global 전역변수1
    전역변수1 변경
    ...
```

예제 9.11 전역변수를 사용하는 함수 예

```
count = 0 # 전역변수 선언
def incremn():
    global count
    count += 10 # 전역변수 count 수정

increment()  # 함수 호출
print(count)
```

실행결과

```
10
```

9.4 재귀함수(Recursive function)

- 재귀함수란 자기 자신을 호출하는 함수이다

- 함수 내부에서 다시 그 함수 자체를 호출하여 반복적인 작업을 수행하는 방식이다

- 재귀함수는 일반적으로 문제를 더 작은 단위로 분해하여 해결하는 데 사용되며, 이 과정을 통해 복잡한 문제를 간결하게 표현할 수 있다

-재귀함수의 성 요소로는 종료 조건(재귀 호출을 멈추는 조건)과 재귀 호출(자기 자신을 호출하는 부분)이 있다.

재귀함수 구조

```
def 함수 이름(매개변수):
    if 종료 조건:
        return 종료 값
    else:
      return 자기 자신을 호출(매개변수 수정)
```

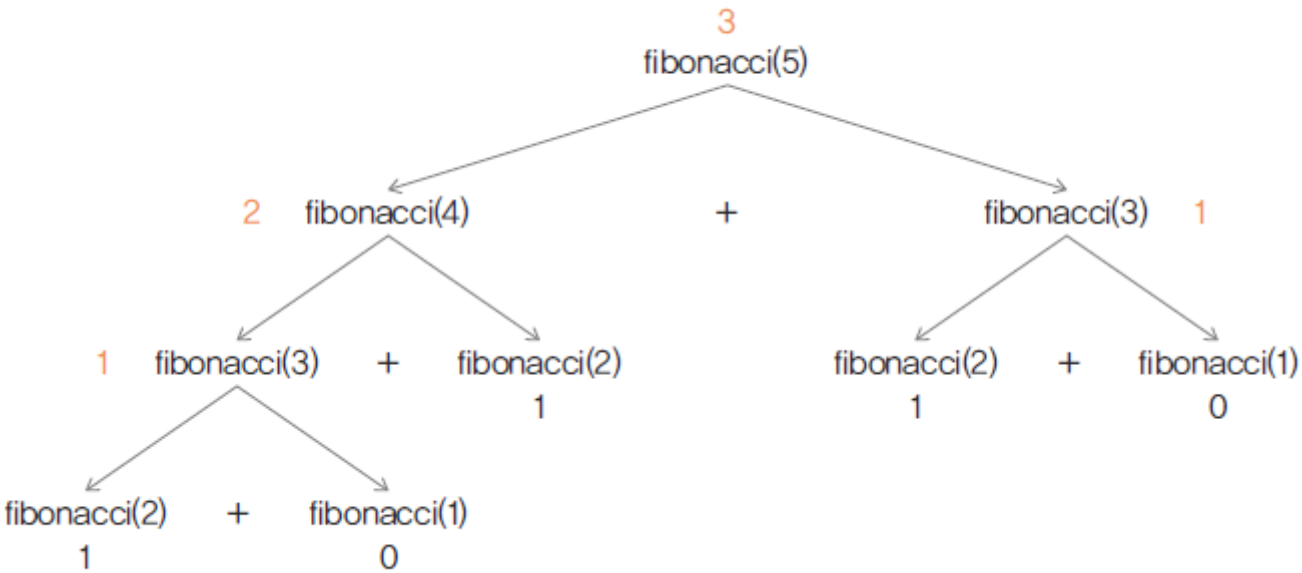
실행결과

```
10
```

9.4 재귀함수(Recursive function)

피보나치 수열 재귀함수의 예

```
def fibonacci(n):
    if n == 1:
        return 0  # f(1)의 값은 0
    elif n == 2:
        return 1  # f(2)의 값은 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)  # 재귀 호출
```



수고하셨습니다.